

ENSF 380

TOPIC 17: CLASS RELATIONSHIPS

Review: Relationships Between Classes

Association

Aggregation

Composition

Dependency

Scenario

House has a roof, kitchen, bathroom, front door, back door

Optional deck

Renovations to add a bathtub and upgrade the sink

Replace the roof and front door

Add a deck

Find tenants and create a rental agreement

Print agreement for lawyer

```
...
public static void main(String[] args) {

    //Build a new house.
    House newBuild = new House("Pat Neil", "123 Main St. NW", "cedar", "2010", 10.0,
+   6, 3, 4.5, "inset");

    //Purchase a new door.
    Door newDoor = new Door("metal", "green");

    //Purchase a new bathtub.
    Bathtub newTub = new Bathtub("clawfoot", "blue");

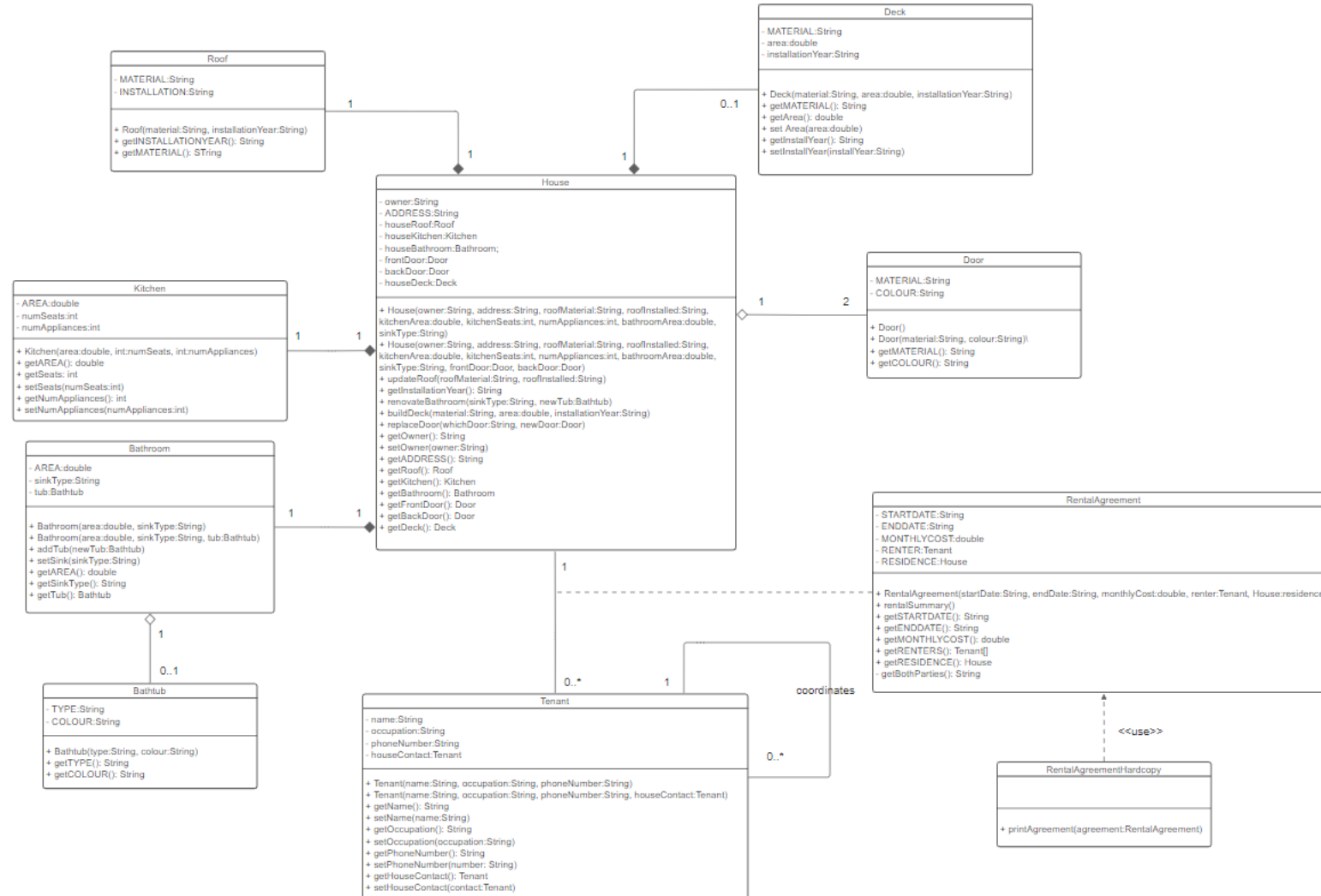
    //Make renovations.
    newBuild.buildDeck("wood", 37.5, "2025");
    newBuild.renovateBathroom("farmhouse", newTub);
    newBuild.updateRoof("asphalt", "2025");
    newBuild.replaceDoor("front", newDoor);
...
}
```

```
...
//Find tenant(s).
Tenant renter1 = new Tenant("Jordan Upton","engineer","587-289-1111");
Tenant renter2 = new Tenant("Jamie Upton","teacher","403-787-2222",renter1);
Tenant[] renterInfo = new Tenant[]{renter1, renter2};

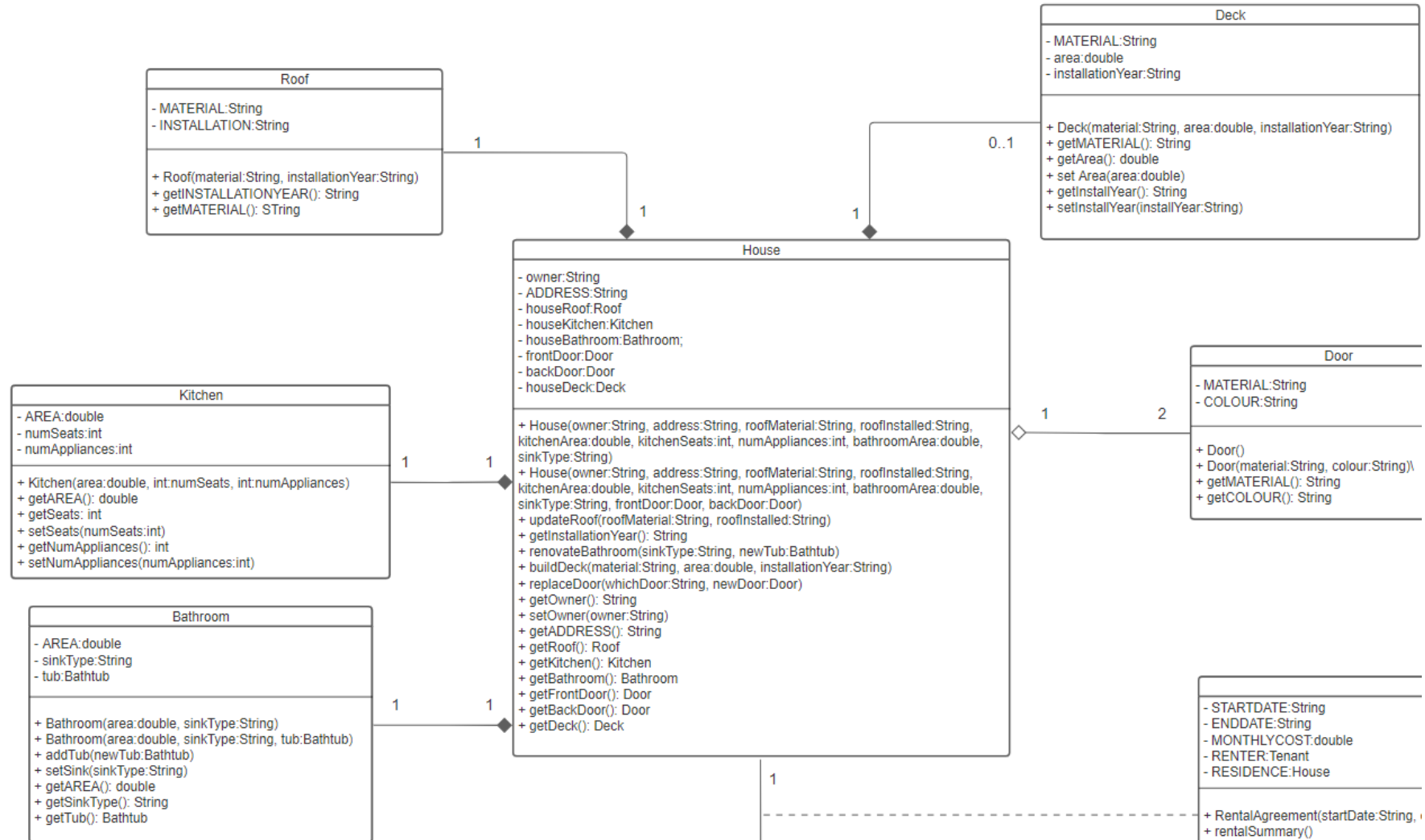
//Sign rental agreement.
RentalAgreement contract = new RentalAgreement("January 2025", "June 2025",
2500.00, renterInfo, newBuild);
contract.rentalSummary();
RentalAgreementHardcopy legalCopy = new RentalAgreementHardcopy();
legalCopy.printAgreement(contract);
}
...
```

UML Diagram

01_RelationshipsExample



Composition: UML



Composition : Code

01_RelationshipsExample

```
...
private String owner;
private final String ADDRESS;
private Roof houseRoof;
private Kitchen houseKitchen;
private Bathroom houseBathroom;
private Door frontDoor;
private Door backDoor;
private Deck houseDeck;

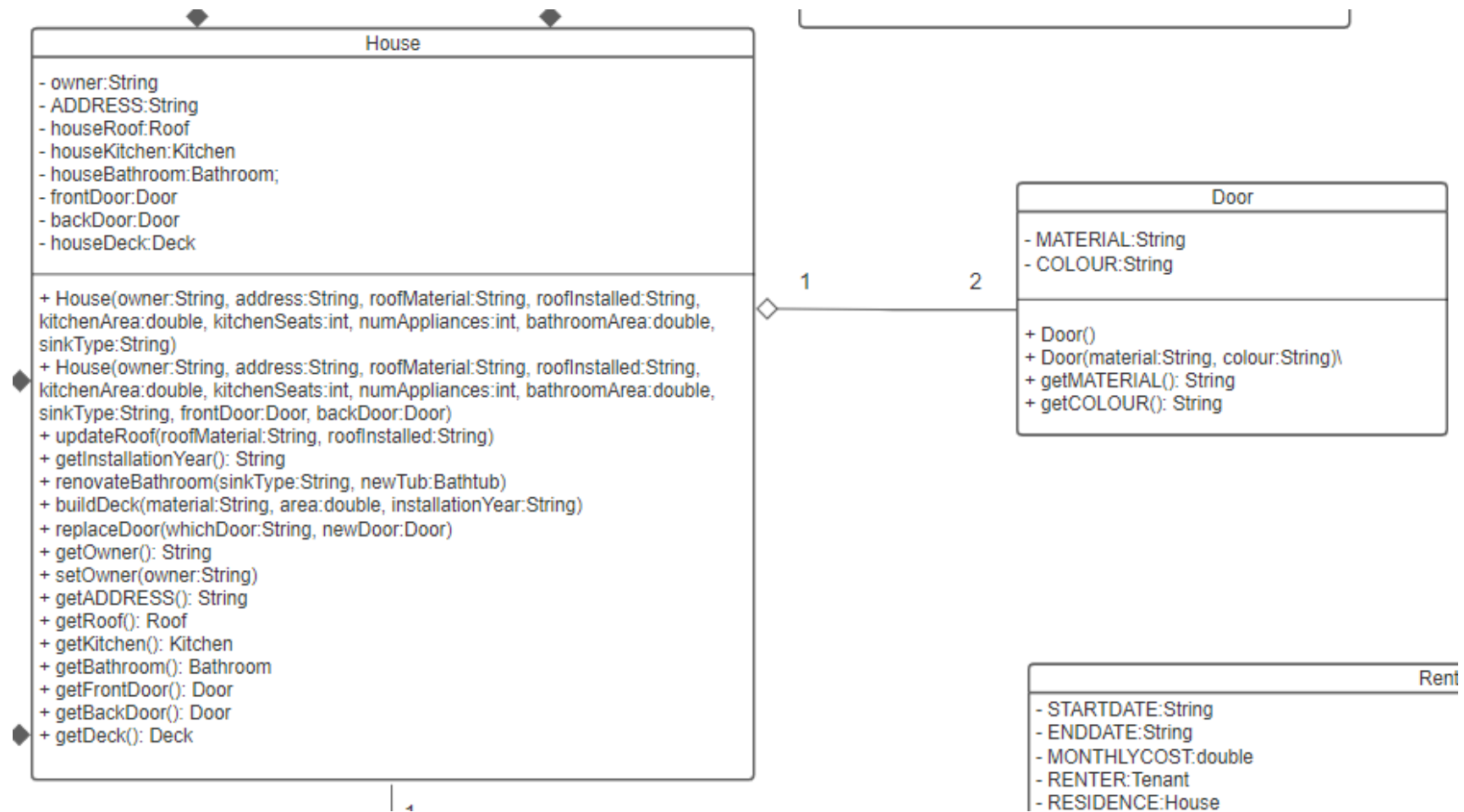
+
public House(String owner, String address, String roofMaterial, String roofInstalled, double
kitchenArea, int kitchenSeats, int numAppliances, double bathroomArea, String sinkType){
    this.owner = owner;
    this.ADDRESS = address;
    this.houseRoof = new Roof(roofMaterial, roofInstalled);
    this.houseKitchen = new Kitchen(kitchenArea, kitchenSeats, numAppliances);
    this.houseBathroom = new Bathroom(bathroomArea, sinkType);
    this.frontDoor = new Door();
    this.backDoor = new Door();
}
```


Composition : Code

01_RelationshipsExample

```
...  
  
public void updateRoof(String roofMaterial, String roofInstalled){  
    this.houseRoof = new Roof(roofMaterial, roofInstalled);  
}  
  
public String getInstallationYear(){  
    return this.houseRoof.getINSTALLATIONYEAR();  
}  
  
public void renovateBathroom(String sinkType, Bathtub newTub){  
    this.houseBathroom.addTub(newTub);  
    this.houseBathroom.setSink(sinkType);  
}  
  
public void buildDeck(String material, double area, String installationYear){  
    this.houseDeck = new Deck(material, area, installationYear);  
}  
...
```

Aggregation: UML



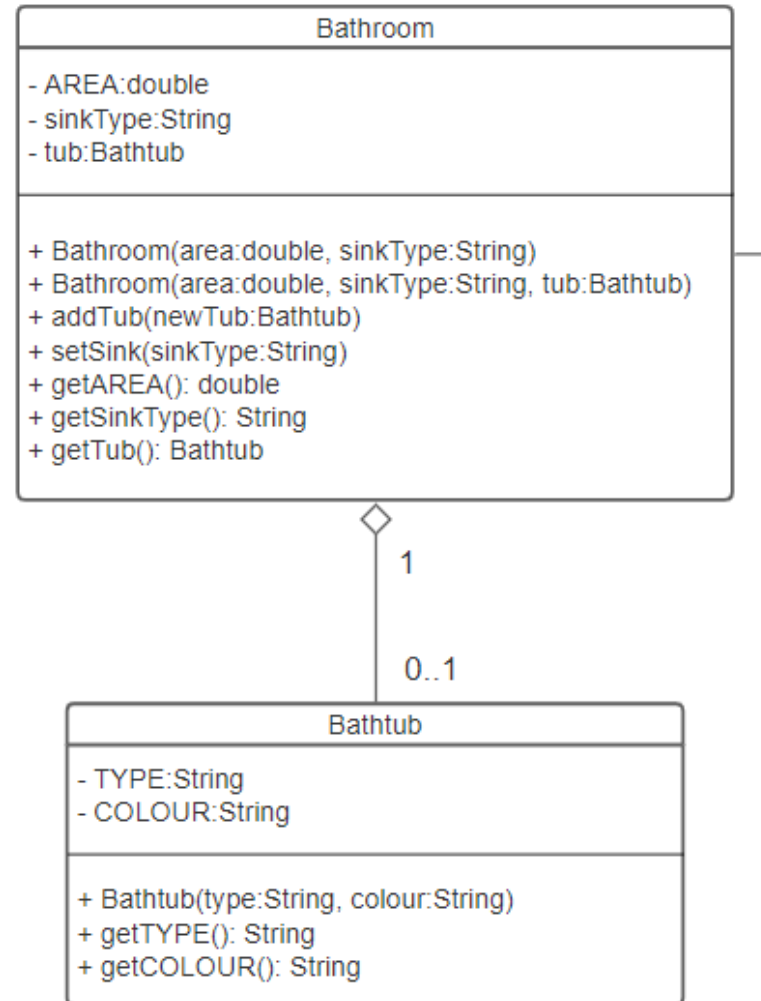
Aggregation: Code

01_RelationshipsExample

```
public House(String owner, String address, String roofMaterial, String
roofInstalled, double kitchenArea, int kitchenSeats, int numAppliances, double
bathroomArea, String sinkType, Door frontDoor, Door backDoor){
    this.owner = owner;
    this.ADDRESS = address;
    this.houseRoof = new Roof(roofMaterial, roofInstalled);
    this.houseKitchen = new Kitchen(kitchenArea, kitchenSeats, numAppliances);
    this.houseBathroom = new Bathroom(bathroomArea, sinkType);
    this.frontDoor = frontDoor;
    this.backDoor = backDoor;
}

public void replaceDoor(String whichDoor, Door newDoor){
    if(whichDoor.equalsIgnoreCase("front")){
        this.frontDoor = newDoor;
    }else if(whichDoor.equalsIgnoreCase("back")){
        this.backDoor = newDoor;
    }
}
```

Aggregation: UML



Aggregation: Code

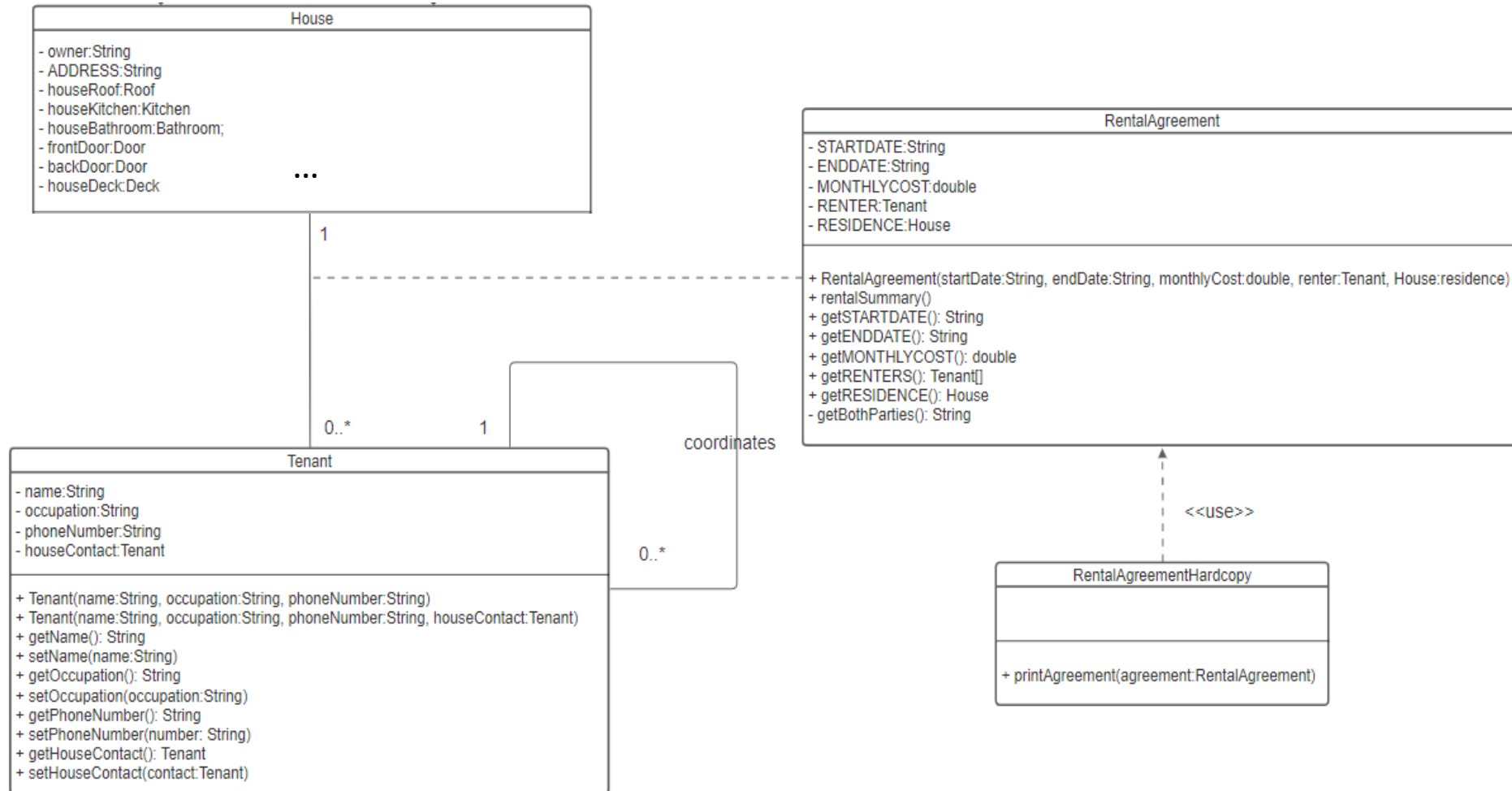
01_RelationshipsExample

```
// Class House
    public void renovateBathroom(String sinkType, Bathtub newTub){
        this.houseBathroom.addTub(newTub);
        this.houseBathroom.setSink(sinkType);
    }

// Class Bathroom
    public void addTub(Bathtub newTub){
        this.tub = newTub;
        System.out.println("Enjoy your new tub!");
    }

// Class Bathtub
    public Bathtub(String type, String colour){
        this.TYPE = type;
        this.COLOUR = colour;
    }
```

Association: UML



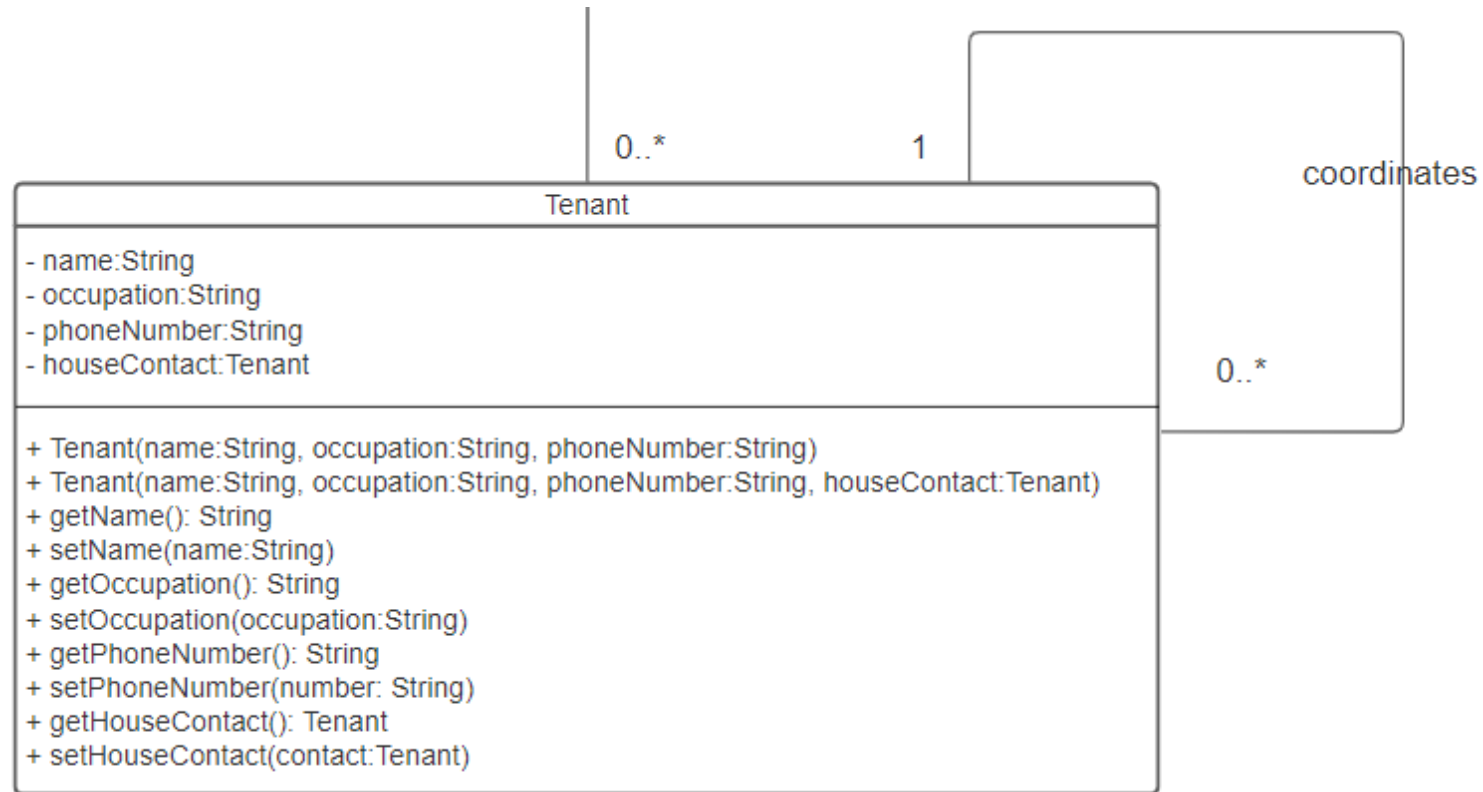
Association: Code

```
class RentalAgreement{

    private final String STARTDATE;
    private final String ENDDATE;
    private final double MONTHLYCOST;
    private final Tenant[] RENTERS;
    private final House RESIDENCE;

    public RentalAgreement(String startDate, String endDate, double monthlyCost,
        Tenant[] renters, House residence){
        this.STARTDATE = startDate;
        this.ENDDATE = endDate;
        this.MONTHLYCOST = monthlyCost;
        this.RENTERS = renters;
        this.RESIDENCE = residence;
    }
}
```

Reflexive Association: UML

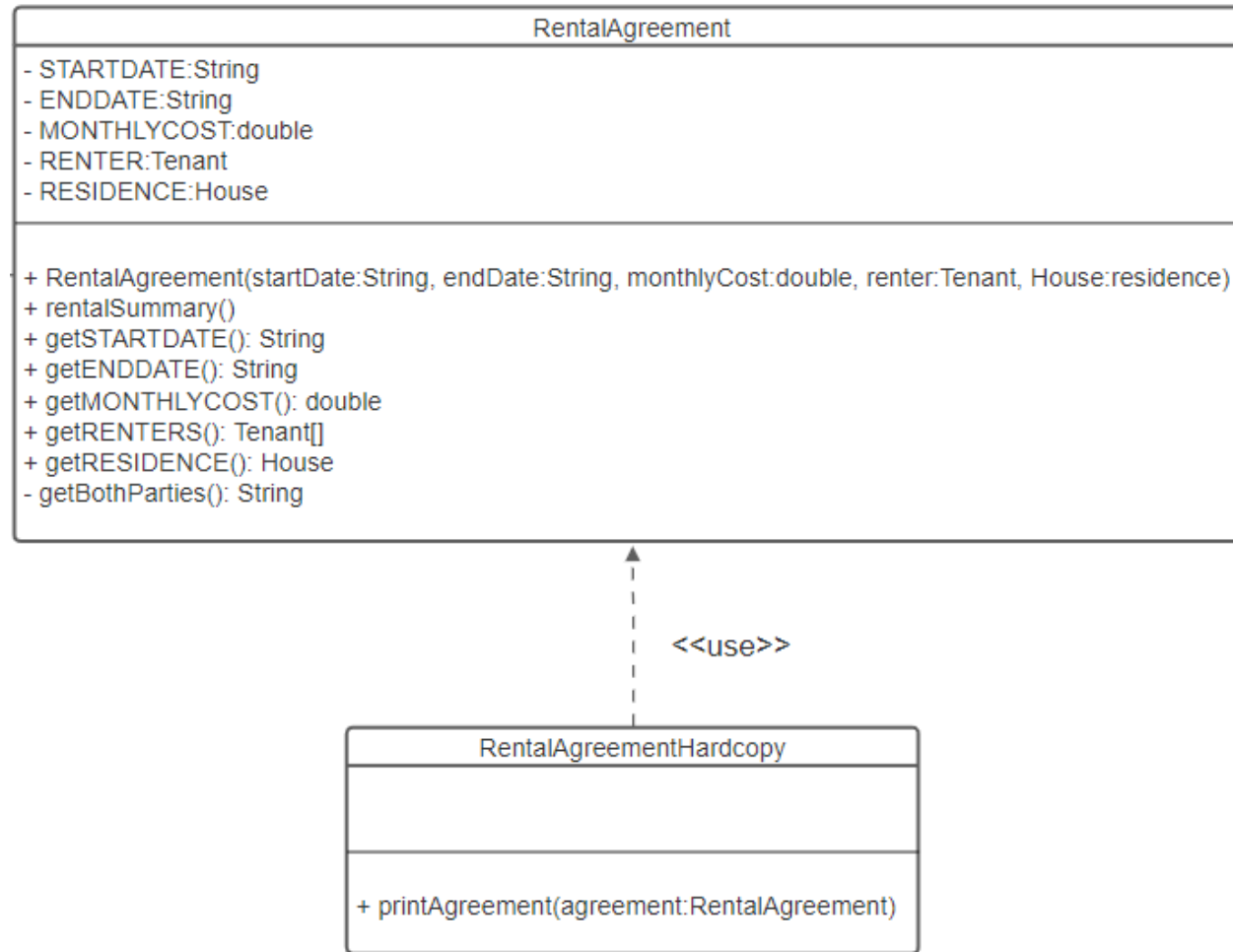


Reflexive Association: Code

01_RelationshipsExample

```
... class Tenant{  
  
    private String name;  
    private String occupation;  
    private String phoneNumber;  
    private Tenant houseContact;  
  
    public Tenant(String name, String occupation, String phoneNumber){  
        this.name = name;  
        this.occupation = occupation;  
        this.phoneNumber = phoneNumber;  
    }  
  
    public Tenant(String name, String occupation, String phoneNumber, Tenant houseContact){  
        this.name = name;  
        this.occupation = occupation;  
        this.phoneNumber = phoneNumber;  
        this.houseContact = houseContact;  
    }  
... }
```

Dependency: UML



Dependency: Code

01_RelationshipsExample

```
... class RentalAgreementHardcopy{  
    public void printAgreement(RentalAgreement agreement){  
        // Not implemented...  
        // Use file I/O and send to PDF...  
    }  
}
```

```
...
public static void main(String[] args) {

    //Build a new house.
    House newBuild = new House("Pat Neil", "123 Main St. NW", "cedar", "2010", 10.0,
+    6, 3, 4.5, "inset");

    //Purchase a new door.
    Door newDoor = new Door("metal", "green");

    //Purchase a new bathtub.
    Bathtub newTub = new Bathtub("clawfoot", "blue");

    //Make renovations.
    newBuild.buildDeck("wood", 37.5, "2025");
    newBuild.renovateBathroom("farmhouse", newTub);
    newBuild.updateRoof("asphalt", "2025");
    newBuild.replaceDoor("front", newDoor);
...
}
```

```
...
//Find tenant(s).
Tenant renter1 = new Tenant("Jordan Upton","engineer","587-289-1111");
Tenant renter2 = new Tenant("Jamie Upton","teacher","403-787-2222",renter1);
Tenant[] renterInfo = new Tenant[]{renter1, renter2};

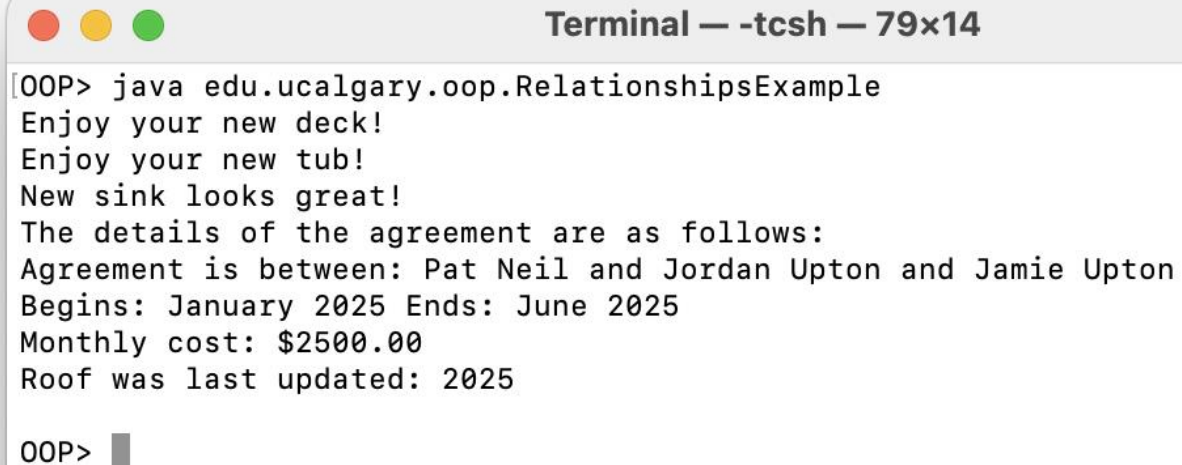
//Sign rental agreement.
RentalAgreement contract = new RentalAgreement("January 2025", "June 2025",
2500.00, renterInfo, newBuild);
contract.rentalSummary();
RentalAgreementHardcopy legalCopy = new RentalAgreementHardcopy();
legalCopy.printAgreement(contract);
}
...
```

Main

01_RelationshipsExample

```
//Find tenant(s).
Tenant renter1 = new Tenant("Jordan Upton","engineer","587-289-1111");
Tenant renter2 = new Tenant("Jamie Upton","teacher","403-787-2222",renter1);
Tenant[] renterInfo = new Tenant[]{renter1, renter2};

//Sign rental agreement.
RentalAgreement contract =
2500.00, renterInfo, newBu
contract.rentalSummary();
RentalAgreementHardcopy le
LegalCopy.printAgreement(c
}
```



```
Terminal — tcsh — 79x14
OOP> java edu.ucalgary.oop.RelationshipsExample
Enjoy your new deck!
Enjoy your new tub!
New sink looks great!
The details of the agreement are as follows:
Agreement is between: Pat Neil and Jordan Upton and Jamie Upton
Begins: January 2025 Ends: June 2025
Monthly cost: $2500.00
Roof was last updated: 2025

OOP> █
```

Exercise 17.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Association

- Association class
- Reflexive association

Aggregation

Composition

Dependency

Summary: Skills

Java implementation of association

Java implementation of aggregation

Java implementation of composition

Java implementation of dependency